

Generative AI with IBM Cloud

Project Documentation format

1. Introduction

- **Project Title:** SmartSDLC – AI-Enhanced Software Development Lifecycle
 - **Team ID:** LTVIP2025TMID32230
 - **Team Leader:** Keerthana Pamula
 - **Team Member:** Telanakula Keerthi Navya
-

2. Project Overview

- **Purpose:**
SmartSDLC is a Generative AI-powered application that enhances and automates core phases of the Software Development Lifecycle. It simplifies requirement analysis, automates backend/frontend code generation, and produces relevant test cases from project inputs.
 - **Features:**
 - Upload PDF or enter prompt to analyze software requirements
 - Generate frontend/backend code using selected languages and frameworks
 - Create unit and functional test cases automatically
 - Easy-to-use Streamlit web interface
-

3. Architecture

- **Frontend:**
Built using **Streamlit**, enabling interactive tabs for Requirement Analysis, Code Generation, and Test Case Generation.
 - **Backend:**
Developed with **FastAPI**, it handles logic processing, PDF handling, and integration with Hugging Face APIs.
 - **Model:**
Uses Hugging Face's **IBM Granite 4.0 Tiny** LLM model for inference tasks like requirement extraction, code generation, and test case creation.
-

4. Setup Instructions

- **Prerequisites:**
 - Python 3.10+

- pip
- Hugging Face Token (for IBM Granite model access)

- **Installation:**

bash

CopyEdit

```
git clone https://github.com/yourusername/SmartSDLC.git
```

```
cd SmartSDLC
```

```
python -m venv venv
```

```
venv\Scripts\activate # For Windows
```

```
pip install -r requirements.txt
```

- **Environment Variables:**
Create a .env file and add:

ini

CopyEdit

```
HF_TOKEN=your_huggingface_token_here
```

5. Folder Structure

bash

CopyEdit

```
SmartSDLC/
```

```
|
```

```
|— app/
```

```
| |— logic.py
```

```
| |— prompts.py
```

```
|
```

```
|— frontend/
```

```
| |— streamlit_app.py
```

```
|— uploads/
```

```
|— .env
```

```
|— requirements.txt
```

6. Running the Application

- **Start Streamlit app:**

bash

CopyEdit

```
streamlit run frontend/streamlit_app.py
```

- App will open at: <http://localhost:8501>
-

7. API Documentation (*Optional for local model use*)

If using FastAPI routes instead of direct logic import, document them like:

bash

CopyEdit

POST /analyze-requirements

POST /generate-code

POST /generate-tests

8. Authentication

No user authentication is required for this version. All features are accessible directly through the UI.

9. User Interface

- Tabbed interface with three key features:
 - Requirement Analysis
 - Code Generation
 - Test Case Generator
 - Upload field, dropdowns for language & framework, and text areas for prompt input.
-

10. Testing

- AI-generated test cases using IBM Granite LLM based on user-provided input (code or requirements).
 - No manual testing framework used; test output is auto-generated in natural language or code.
-

11. Screenshots or Demo

(Add screenshots of Streamlit interface and outputs here)

- Interface showing PDF upload
 - Sample generated code
 - Generated test cases
-

12. Known Issues

- Model responses may vary slightly across prompts
 - Slower on first run due to model loading time
 - Requires valid Hugging Face token for inference
-

13. Future Enhancements

- Add user login & project history tracking
- Support for uploading multiple PDFs
- Expand language/framework coverage
- Enable saving test reports and code outputs locally or to the cloud